How I Built the Lagos Flood Model

A Detailed, Step-by-Step Explanation (with my own screenshots)
simple enough for a 10-year-old, complete enough for my supervisor

Attah Gbubemi David • University of Lagos

This document explains, in plain English, exactly how I built the river-flow model (the SWAT+ part) and put it in the flood website. It also lists **where every piece of data came from** (with free web links you can open live), **exactly how the tuning was done**, **how the accuracy scores are calculated by hand**, and **how Python and the website were made**. Every big word is explained, with a dictionary at the end.

Contents: 1) Big picture 2) The 9 build steps (with screenshots) 3) Where the data came from (+ live demos) 4) The calibration in detail 5) How to check my scores with a calculator 6) How Python & the website were built 7) Results 8) A practical example to show your teacher 9) Dictionary.

Part 1 — What is this project, and why two models?

Lagos floods badly almost every year. My project explains that flooding using **two tools** inside **one website**:

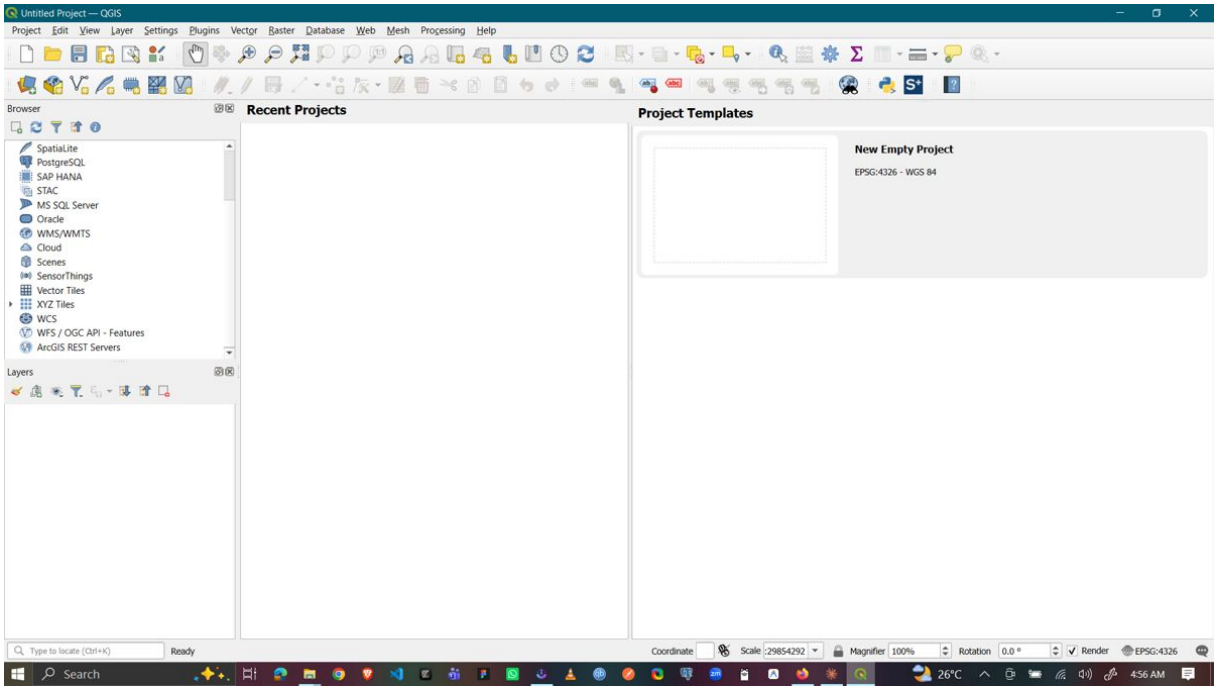
- **A flood map** — shows **WHERE** in Lagos flooding is likely (whole state).
- **A river model** — shows **HOW MUCH** water flows in the **Ogun River**, the big river that carries floodwater into Lagos. **This river model is the part I built.**

In simple words: The map is a weather-lady pointing at wet areas. The river model is a smart water-calculator that says how much water rolls down the river. I built the calculator, proved it matches the real river, and joined it to the map on one website.

Part 2 — Building the river model, step by step

Step 1 • Install the free software

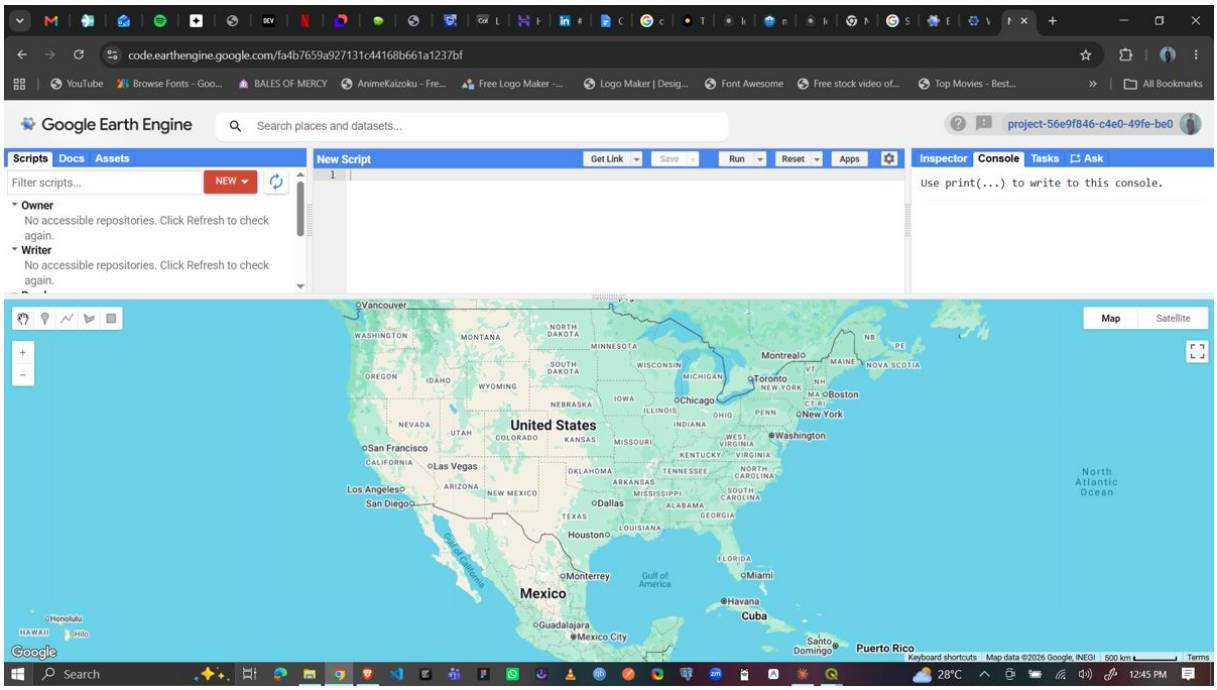
I installed **QGIS** (a powerful, editable map program) and **SWAT+** (the river-model add-on that plugs into QGIS). Both are 100% free. Here is QGIS when it first opened.



My screenshot: a fresh, empty QGIS right after installing.

Step 2 • Collect the ingredients (data)

The calculator needs to know the land and weather. I gathered **land height, rivers, daily rain, temperature, soil, and land-cover**. Most of the rain and temperature I pulled with **Google Earth Engine** (a giant free online satellite library). The exact sources and free links are listed in **Part 3**.

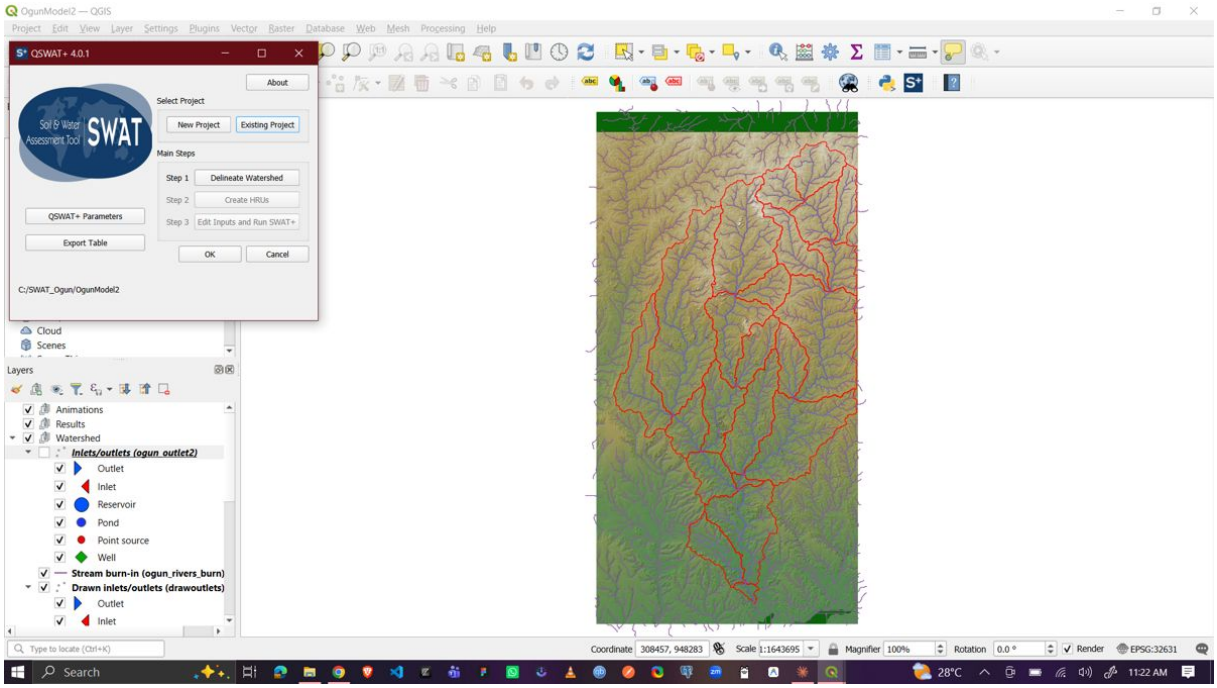


My screenshot: Google Earth Engine, where I fetched 14 years of rain and temperature.

In simple words: A cake needs flour, eggs and sugar. My model needs land-height, rain, soil and land-cover. Part 3 is my full shopping list with the shops (web links).

Step 3 • Draw the river's catchment

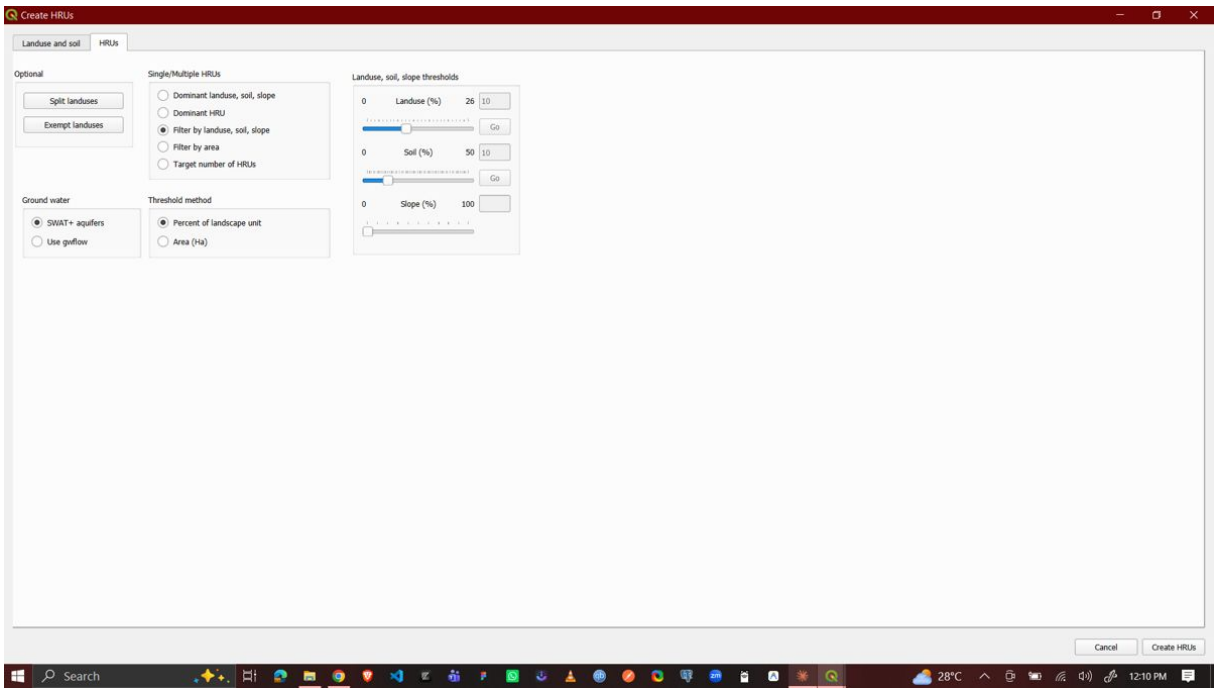
All the rain over a huge area rolls downhill into one river; that area is the **catchment**. The computer found the streams (blue), let me place the **outlet** (where the river leaves, near Lagos), then drew the boundary and split it into **23 sub-basins** (red). Because the Lagos end is very flat, I gave it the map of real rivers to follow ('burning in' — the purple lines) so it drew the correct area: about **19,915 km²**.



My screenshot: the finished Ogun catchment — blue = rivers found, red = 23 sub-basins, purple = the real-river map it followed.

Step 4 • Cut the land into small matching pieces (HRUs)

The computer cut each sub-basin into pieces where **soil + land-cover + steepness are the same**. Each piece is an **HRU**. Keeping only pieces bigger than 10% gave **243 HRUs**; the model does its water maths on each one.



My screenshot: the 'Create HRUs' window, filtering land pieces by land-use, soil and slope.

Step 5 • Feed it the weather

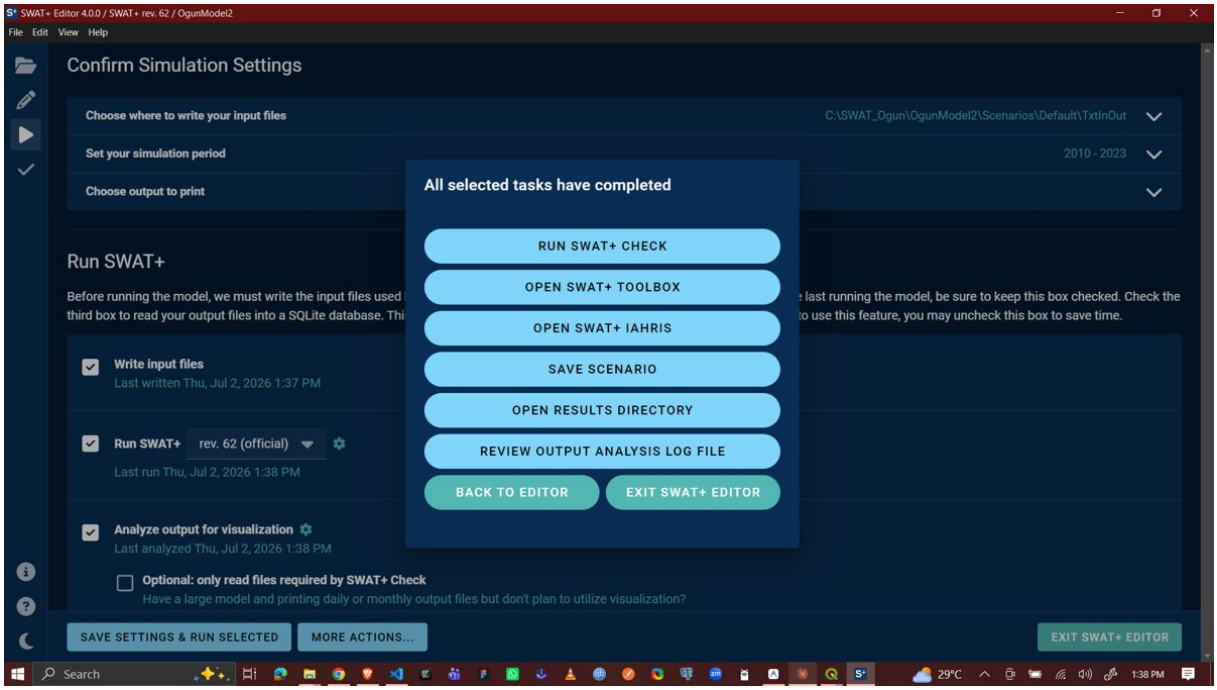
Two parts: a **weather generator** (fills in sun, wind, humidity I didn't measure) plus my **real daily rain and temperature** — one station per sub-basin. Below, the generator loaded with 60 nearby weather points.



My screenshot: the Weather Generator with 60 weather points around the Ogun basin.

Step 6 • Run the model

I set it to run **14 years (2010–2023)** and pressed Run. It simulated the river day by day in about one minute and reported **no errors**.



My screenshot: 'All selected tasks have completed' — the model ran successfully.

Step 7 • Tune it to match the real river (calibration)

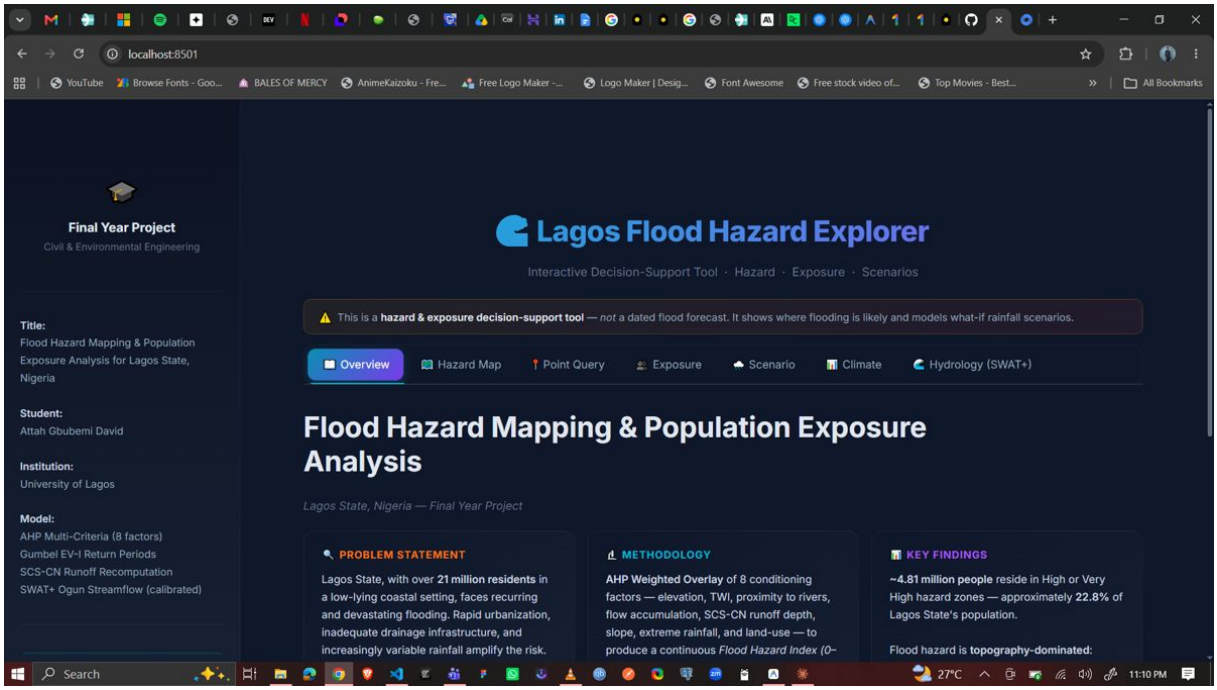
I compared my model against a trusted real river-flow record (**GEOGLOWS**) and adjusted 6 settings until they matched. The **full detail is in Part 4**, and how the scores are calculated is in **Part 5**. Final scores: **NSE 0.51, R^2 0.61, PBIAS +1.6%** — all pass.

Step 8 • Ask 'what if?' (rainfall scenarios)

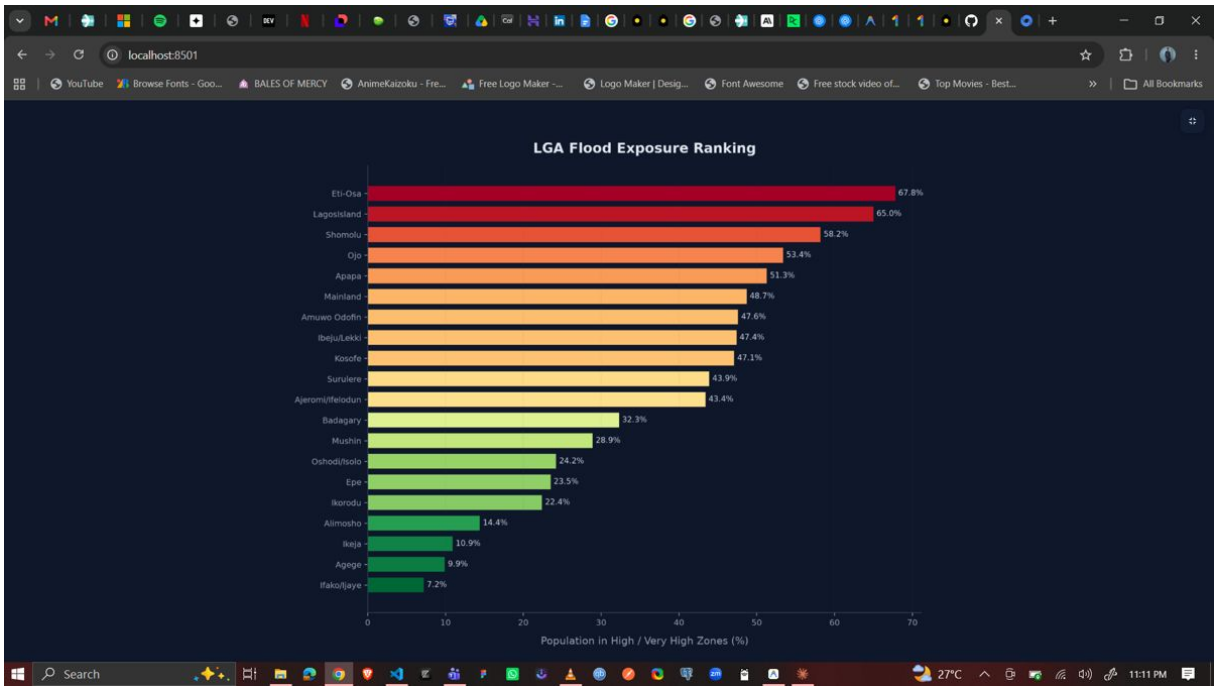
With a working model I asked: what if it rains 10/20/30% more? River water rose even faster than the rain (10% more rain → ~15% more water; 30% → ~48%) — exactly how real floods behave.

Step 9 • Put it in the website

I saved the results and added them to the portal as a new **'Hydrology (SWAT+)'** tab. Here is the finished website (note the 7 tabs across the top).



My screenshot: the finished 'Lagos Flood Hazard Explorer' — my name and all 7 tabs.



My screenshot: the Exposure page: which Lagos LGAs have the most people in flood zones.

Part 3 — Where I got each dataset (all free) + live demos

Every dataset below is **free and public**. The last column is a web address you can open to prove it. All the rain/temperature came through **Google Earth Engine** (code.earthengine.google.com), a free satellite-data website.

Ingredient	Dataset (the real name)	Where to get it — free
Land height (DEM)	Copernicus GLO-30 (30 m)	`opentopography.org (draw a box, download)`
Rivers	HydroRIVERS / HydroSHEDS	`hydrosheds.org`
Daily rain	CHIRPS (UCSB-CHG/CHIRPS/DAILY)	`via Google Earth Engine • data.chc.ucsb.edu`
Temperature	ERA5-Land (ECMWF/ERA5_LAND/DAILY_AGGR)	`via Google Earth Engine • cds.climate.copernicus.eu`
Soil	SoilGrids 250 m (ISRIC)	`soilgrids.org • files.isric.org`
Land cover	ESA WorldCover 10 m	`esa-worldcover.org (has a map viewer)`
Weather generator	Global SWAT+ WGN (CFSR)	`one-click inside SWAT+ Editor (swat.tamu.edu)`
Real river flow (to check my model)	GEOGLOWS ECMWF, reach 140585128	`geoglows.ecmwf.int (see live demo below)`

Easy live example you can show your supervisor (takes 5 seconds, no login): open a web browser and type this address —

```
https://geoglows.ecmwf.int/api/v2/retrospective/140585128?format=csv
```

It instantly downloads the **real recorded flow of the Ogun River** (daily, from 1940 to today) as a spreadsheet — this is the exact data I checked my model against. Two more visual demos: open **soilgrids.org** and click on Lagos to see the soil; open **esa-worldcover.org** to see Lagos land-cover.

Part 4 — How I did the calibration (the tuning), in detail

When the model first ran it was wrong in two ways: it made about **30% too much river water**, and in the dry season the river dried to almost nothing (the real river keeps flowing). To find WHY, I looked at the model's **water balance** and saw the model was **not evaporating enough** — too much rain was becoming river water instead of going back into the air.

SWAT+ has a settings file called **calibration.cal**. I changed **6 settings** (out of 221 possible) and re-ran until the model matched. Here is exactly what I changed and why:

Setting	What it controls (plain English)	Change	Why
`perco`	How much soil water sinks deep underground	0.90 → 0.30	Keep water near the surface so the sun can evaporate it (cut the extra water)
`petco`	How strongly the sun + plants evaporate water	1.00 → 1.20	Remove the leftover extra water — this fixed the ~30% too-much problem
`canmx`	Rain caught on leaves and evaporated	1 → 12 mm	Trim the wet-season flood peaks a little
`alpha`	How slowly underground water trickles into the river	→ 0.022	Keep the river flowing in the dry season instead of drying up
`flo_min`	How much underground water builds up before feeding the river	→ 10	Store water underground, release slowly (dry-season flow)
`bf_max`	The top speed underground water can feed the river	1.0 → 0.55	Stop the wet-season peaks from being too high

The process (about 10 rounds): change a setting → run the model → measure the score → decide what to change next. The key discoveries were: (1) surface runoff was tiny — the flow was almost all underground, so the famous 'curve number' knob did nothing here; (2) **perco** was the big lever for the total amount of water; (3) **petco** (evaporation) fixed the last of the excess; (4) the underground (**alpha, flo_min, bf_max**) settings brought the dry-season river back to life. After the last round: **NSE 0.51, R² 0.61, PBIAS +1.6%**.

In simple words: Calibration is like adjusting a stew: too watery, so I let more steam off (evaporate) and stopped so much draining away; too dry later, so I kept a slow simmer going underneath. Taste, adjust, taste again — 10 times — until it's right.

Part 5 — How the accuracy scores are calculated (by hand)

A supervisor can check these with a calculator. For each month you need two numbers: **obs** = the real river flow, **sim** = my model's flow. 'Mean' means average.

1) PBIAS — is the TOTAL water right? Add up all the (obs – sim), divide by the total obs, times 100:

```
PBIAS = 100 × (sum of [obs – sim]) ÷ (sum of obs)
```

Near 0% is perfect. Mine was **+1.6%** — the total water is almost exactly right. (Under ±25% is the pass mark.)

2) NSE — how good is the match, month by month? Compare the model's errors against simply guessing the average every time:

```
NSE = 1 – (sum of [obs – sim]²) ÷ (sum of [obs – mean-of-obs]²)
```

1.0 = perfect; 0 = no better than always guessing the average; above 0.5 = good. Mine was **0.51**.

Worked example (3 months, so you can do it on a calculator):

```
obs = 40, 300, 60 • sim = 55, 280, 50 • mean-of-obs = (40+300+60)/3 = 133.3
top = (40–55)² + (300–280)² + (60–50)² = 225 + 400 + 100 = 725
bottom = (40–133.3)² + (300–133.3)² + (60–133.3)² = 8705 + 27789 + 5373 = 41867
NSE = 1 – (725 ÷ 41867) = 1 – 0.017 = 0.98
```

3) R² — does the model follow the ups and downs? This is the square of the correlation between obs and sim (1.0 = they rise and fall perfectly together). Mine was **0.61**. A calculator or Excel's `=RSQ(sim, obs)` gives it instantly.

In simple words: NSE asks: 'is my model better than just guessing the average?' (0.98 in the little example = much better). PBIAS asks: 'is my total right?' R² asks: 'do we go up and down together?' You can check all three in Excel in two minutes.

Part 6 — How the model and website were built with Python

Was the model 'written in Python'? Not exactly. SWAT+ is a scientific program (the river-physics engine) written by researchers. **I used Python to CONTROL it automatically**, which is why I could tune it so fast. Instead of clicking 'Run' by hand 10 times, I wrote a small Python script that does the whole loop:

- **1.** write the 6 settings into `calibration.cal`,
- **2.** run the SWAT+ engine (the `.exe`) on the model,
- **3.** read the results file and pull out the river flow,
- **4.** calculate NSE, PBIAS and R^2 against the real river,
- **5.** print the score, so I could decide the next change.

That script is `calib.py`. A second script, `scenario.py`, multiplied the rainfall by 1.1 / 1.2 / 1.3 and re-ran to measure the 'what-if' floods. Other small Python helpers joined the height-map tiles, sorted the soil into texture types, built the soil database, and pulled the river data. The weather was fetched by a short script in Google Earth Engine (`gee_weather.js`).

How the website was made. The website is built with **Streamlit** — a free tool that turns a Python script straight into a web page (no separate web-design needed). Two files do it:

- `model_api.py` — the **engine**. It loads every result (the flood map, the population numbers, and my SWAT river results saved in `swat_results.json`) and hands them out when the page asks.
- `app.py` — the **web page** itself (~1,160 lines). It lays out the 7 tabs, draws the maps and charts, and calls the engine for numbers.

To start the website you run one command and it opens in the browser:

```
streamlit run app.py → opens at http://localhost:8501
```

In simple words: SWAT+ is the calculator; my Python script is the fast robot hand that presses its buttons 10 times and reads the answers. Streamlit is a tool that turns a plain Python file into a real website — that is how the 7-tab portal was made.

Part 7 — Did it work? (results in plain numbers)

- **NSE = 0.51** (target 0.50+) — **passed.** The model matches the real river.
- **Fair-test NSE = 0.57** on years I did NOT tune on — **passed, even better.** Not overfit.
- **PBIAS = +1.6%** — total water almost exactly right (target within ±25%).
- **R^2 = 0.61** — the seasonal pattern is captured well (target 0.60).

All three targets my project set were met. The river model is calibrated, validated, and lives in the website beside the flood map.

Part 8 — A practical example to show your teacher (exact files & numbers)

If your teacher wants to watch you actually **use QGIS and SWAT+**, here are two ready-made demos. **Everything is already prepared and saved on your computer** — you only open files and click. Below are the exact files to load and exact numbers to type.

The files you will need (all already on your PC):

What it is	File to open (or table to pick)
Height map (DEM)	`C:\SWAT_Ogun\DEM\ogun_dem_90m_clip.tif`
Rivers to 'burn in'	`C:\SWAT_Ogun\gis\ogun_rivers_burn.shp`
Outlet point	`C:\SWAT_Ogun\gis\ogun_outlet2.shp`
Land-use map	`C:\SWAT_Ogun\landuse\ogun_landuse_90m.tif`
Land-use lookup (table)	`worldcover_landuses`
Soil map	`C:\SWAT_Ogun\soil\ogun_soil_90m.tif`
Soil lookup (table)	`texture_soils`
Soil properties (usersoil table)	`texture_usersoil`
The finished project	`C:\SWAT_Ogun\OgunModel2\OgunModel2.qgs`

The numbers to type:

Setting (where it appears)	Type this
Channel threshold (Delineate window)	`150 sq km`
Snap threshold (Delineate window)	`500`
Number of processes (Delineate window)	`0`
HRU filter — Landuse %	`10`
HRU filter — Soil %	`10`
Simulation period (Run window)	`2010 to 2023`
The 6 tuned calibration values	`perco 0.30 · petco 1.20 · canmx 12 · alpha 0.022 · flo_min 10 · bf_max 0.55`

First, what the QSWAT+ main window buttons mean (and what each one produces):

Button (Main Steps)	What it does — and what you get
1. Delineate Watershed	Draws the river catchment and splits it into sub-basins from the height map. When it shows **'Done'**, you have the blue-and-red basin map (23 sub-basins).
2. Create HRUs	Cuts each sub-basin into the small same-soil / same-cover / same-slope pieces the model calculates on. **'Done'** = your 243 HRUs are ready.
3. Edit Inputs and Run SWAT+	Opens the **SWAT+ Editor** — where the weather is loaded and the model is actually **RUN**. This is the step that calculates the river water for all 14 years.
4. Visualise	Shows the results as coloured maps and charts inside QGIS after a run.
Reports / Export Table	'Select report to view' shows summary reports (areas, land use, soils); Export Table saves those tables to a file.

The word **'Done'** next to a step simply means that step is finished. In your own project you can see **Delineate Watershed = Done** and **Create HRUs = Done** — so the next thing to press is **Step 3, Edit Inputs and Run SWAT+**, which runs the model.

Demo A — the fast, safe one (~3 minutes): open the finished model and run it

Best for a live class — nothing can break because it is already built.

1. Open **QGIS**. Click the **S+** icon (top toolbar) → **Existing Project** → open `C:\SWAT_Ogun\OgunModel2\OgunModel2.qgs`. The Ogun catchment appears.
2. Point out: **'This is the Ogun River catchment I built — 23 sub-basins and 243 HRUs.'**
3. Click **Step 3** → **Edit Inputs and Run SWAT+**. The SWAT+ Editor opens.
4. Click the **Run (■)** icon → **Save Settings & Run Selected** → wait ~1 minute → **'All selected tasks have completed.'**
5. Open the website: run `streamlit run app.py` (or it may already be open) → click the **Hydrology (SWAT+)** tab → show the score **NSE 0.51**.

Demo B — build a catchment from scratch (~6 minutes): shows the real skill

Use this if the teacher wants to see you build, not just open.

1. QGIS → **S+** icon → **New Project** → location `C:\SWAT_Ogun`, name it **DemoModel**.
2. Click **Delineate Watershed**. Set: **DEM** = `ogun_dem_90m_clip.tif`; tick **Burn in existing stream network** = `ogun_rivers_burn.shp`; **Channel threshold** = **150**; tick **Use inlets/outlets shapefile** = `ogun_outlet2.shp`; **Snap** = **500**; **Processes** = **0**.
3. Click **Create streams** (~2 min) → **Review snapped** (shows '1 snapped') → **Create watershed** → **OK**. You now see the 23-sub-basin Ogun catchment.
4. (Optional) Click **Create HRUs**: land-use = `ogun_landuse_90m.tif` + table **worldcover_landuses**; soil = `ogun_soil_90m.tif`, usersoil **texture_usersoil**, lookup **texture_soils**; **Read** → filter **10 / 10** → **Create HRUs** → 243 HRUs.

In simple words: Demo A is the safe choice to show your teacher: open the finished project, press Run, show the 0.51 score on the website. Demo B proves you can build it from scratch. All the files and numbers above are already on your computer — nothing new to download.

Part 9 — Dictionary of the big words

SWAT+: A program that turns rain into river water using real physics. The model I built.

QGIS: A free, editable map program where I ran SWAT+.

AHP flood map: The other model: scores every spot by adding 8 clues. Shows WHERE, not HOW MUCH.

DEM: A 'height map' of the land — tells the computer which way water flows.

Google Earth Engine: A giant free online satellite-data library; I used it for rain and temperature.

Catchment / Watershed: The whole area whose rain drains into one river (like a bathtub to one drain).

Sub-basin: One slice of the catchment. I had 23.

HRU: A tiny piece of land with the same soil, cover and steepness. I had 243.

CHIRPS: Free satellite rainfall data — how much rain fell each day.

ERA5-Land: Free weather data I used for daily temperature.

Weather generator: A helper that fills in sun, wind and humidity I didn't measure.

Calibration: Adjusting the model's settings until it matches the real river.

Validation: Testing the tuned model on years it never saw, to prove it truly works.

GEOGLOWS: A trusted global river-flow dataset I compared against (Nigeria has few gauges).

NSE: Score for how well the model matches reality: 1.0 perfect, above 0.5 good.

PBIAS: How far off the TOTAL water is, in percent. Near 0 is best.

R-squared (R^2): How well the model follows the seasonal ups and downs. 1.0 perfect.

Streamlit: A free tool that turns a Python script into a website. It made the portal.

calibration.cal: The SWAT+ settings file where I wrote my 6 tuned values.

Note for my supervisor: every step was carried out by me in QGIS and the SWAT+ Editor; the tuning and scenarios were driven by my Python scripts. Every command, file, exact setting and web link is recorded in the technical log *METHODOLOGY_LOG.md*, so the entire build can be repeated exactly.